

USE CASES & SECURITY READINESS

The Always-On Work Agent

A persistent agent runtime for durable, tool-connected operational work — and the controls that make it enterprise-ready.

EXECUTIVE SUMMARY

A runtime for agents that own ongoing responsibilities

OpenClaw is best understood as a runtime for long-running, tool-connected agents that own ongoing responsibilities – **not simply another coding assistant.**

It earns its place when a team wants an agent to watch systems, maintain context, respond to events, take bounded actions, and escalate when human approval is required.

THE THROUGH-LINE

The same capabilities that make OpenClaw useful – **persistence, memory, tool access, credentials, external inputs, and autonomy** – are also what create security concerns.

The answer is delegated responsibility inside enforceable boundaries: the capabilities create the value, the platform enforces the limits.

POSITIONING

What OpenClaw is – and is not

IS

- ✓ A platform for persistent agents that observe, reason, act, and report over time
- ✓ A way to delegate bounded operational responsibilities to agents
- ✓ Enterprise-relevant when paired with identity, access control, auditability, isolation, and approval gates

IS NOT

- ✗ A replacement for every coding assistant
- ✗ An unrestricted autonomous employee
- ✗ A generic chatbot with a longer context window
- ✗ A cron job with an LLM attached

WHY OPENCLAW

Most enterprise work is a stream of responsibilities, not isolated tasks

These workflows are persistent, event-driven, and spread across multiple systems. OpenClaw's value is that an agent can be assigned a **durable responsibility** – not just answer a question.

Overnight alert watch & escalation

Morning briefings & standup prep

Triaging & routing incoming issues

Chasing stale PRs & tickets

Watching repos & cluster events

Keeping docs & runbooks current

Tracking action items across meetings

Tracking upstream releases & CVEs

The common thread: responsibilities across systems that no single session can hold.

OpenClaw and Claude Code: different jobs

Dimension	OpenClaw	Claude Code
Primary mode	Persistent, always-on	Interactive, in-session
Best for	Monitoring, memory, automation	Writing & changing code
Duration	Continuous, over time	A focused session
Human involvement	Low – works in background	High – present & steering
Example task	"Brief me on overnight changes & risks."	"Refactor this and fix the test."

BY ROLE

One runtime, role-specific responsibility

DEVELOPER / PLATFORM

Project continuity agent

Repo monitoring, PR briefs, CI failure triage, release notes, architecture memory, safe cluster checks.

MARKETING / COMMS

Campaign continuity

Tracks campaign calendars and deadlines, watches performance and brand mentions, drafts recurring content, remembers what resonated.

FINANCE / OPS

Recurring spend analysis

Monitors budgets and invoices, flags anomalies, tracks renewals, explains variance, remembers approvals.

MANAGER / LEAD

Continuous status

Weekly status & blocker tracking, meeting-note action items, 1:1 prep, continuity through handoffs.

SUPPORT / SUCCESS

Escalation briefing

Summarizes issues, links recurring problems to known bugs, remembers customer environments and incidents.

The common pattern

Hold the context, watch what changes, prepare the work – so people decide and act instead of reassembling.

WHY NOT ANOTHER HARNESS

The question isn't which harness runs on OpenShift – many can

Hermes or similar frameworks could be containerized, connected to tools, and wrapped with platform controls too. That's not the differentiator.

THE REAL QUESTION

Can we define a **repeatable, governable pattern** for running persistent, tool-connected agents in the enterprise?

That takes more than an agent loop – it needs identity, permissions, credentials, isolation, observability, memory governance, approval gates, lifecycle, and audit.

OPENCLAW = REFERENCE HARNESS

Not "automatically enterprise-ready." It's the **concrete runtime we harden, test, document, and integrate** to prove what a secure agent deployment model looks like on OpenShift.

Any other harness would still face the same review – memory, tool loading, credentials, untrusted input, action logging, approvals.

THE OPERATING MODEL

Agent behavior, governed by platform controls

OpenShift provides

THE GOVERNANCE LAYER

Workload isolation

RBAC

Network policy

Image scanning

Service accounts

Secrets management

Observability

Lifecycle controls



OpenClaw provides

THE AGENT BEING GOVERNED

Observe over time

Take bounded action

Report & summarize

Tool connections

Reason on events

Maintain memory

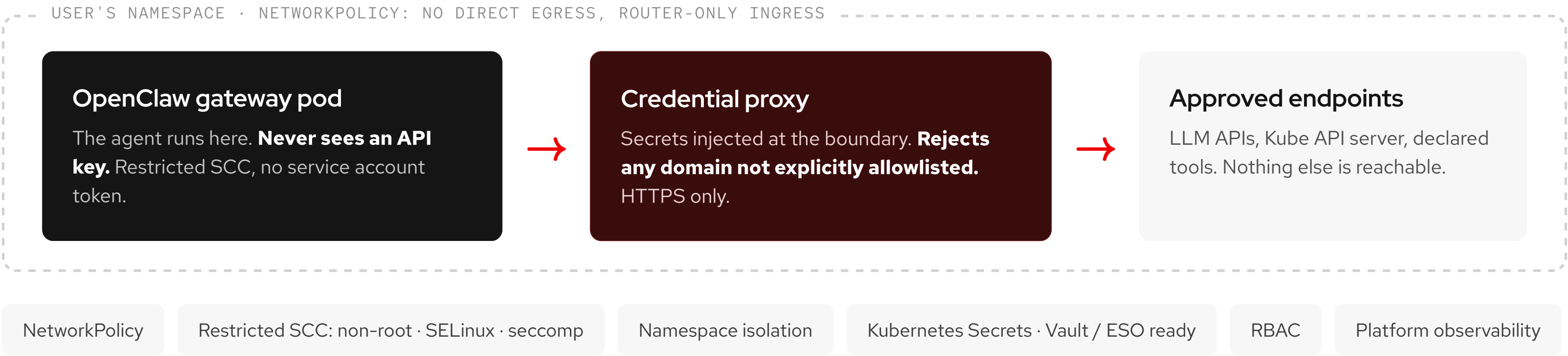
Escalate for approval

Scoped identity

The combined question: **what does the secure operating model for this class of agent look like?**

THE PATTERN IN PRACTICE

OpenShift primitives do the enforcing



Everything that confines the agent to the proxy is stock OpenShift. An operator stamps this pattern out per instance.

ADOPTION MODEL

From observation to bounded action

Each stage has a clear permission boundary and review model – so capability grows only as trust and controls are proven.

01

Read-only

Summarize, classify, and report. No writes.

02

Low-risk writes

Labels, draft comments, internal notes, generated reports.

03

PR-based changes

Code or config changes reviewed before merge.

04

Gated writes

Operational systems – only with narrow RBAC, audit, and rollback.

05

Privileged

Cross-namespace actions – mature workflows, explicit approvals only.

LOWER RISK

MORE CAPABILITY

SECURITY FRAMING

Not unrestricted autonomy. Controlled autonomy.

Delegated responsibility inside enforceable boundaries — observe broadly where appropriate, act narrowly where permitted, escalate where risk is meaningful.



Observe broadly

Read across systems where it's appropriate to build context.



Act narrowly

Take only the specific, permitted actions each workflow allows.



Escalate on risk

Route to human approval whenever an action carries meaningful risk.

Every use case defines: what it can read · write · what needs approval · what is logged · what is intentionally denied · what residual risk remains.

CAPABILITY → RISK → CONTROL

Every capability pairs value with a named mechanism

Capability	Value	Security risk	Control on OpenShift
Persistence	Track work over time	Long-lived access widens blast radius	Namespace-scoped identity + RBAC, periodic permission review, audit logs
Memory	Retain project context	Memory poisoning or stale context	Instance-scoped storage, provenance, reviewable memory updates
Tool access	Take useful action	Unauthorized or unsafe action	Least-privilege RBAC per namespace, approval gates, restricted tools
External inputs	Watch issues, chats, docs, tickets	Prompt injection from untrusted content	NetworkPolicy egress allowlist bounds what injection can reach; treat external text as untrusted
Credentials	Connect to real systems	Secret leakage or misuse	Kubernetes Secrets + proxy injection: the agent never holds keys
Autonomy	Reduce manual toil	Harder accountability	Action logs, platform observability, clear ownership, approval thresholds
Skills / plugins	Extend the agent	Supply-chain risk	Pinned versions, review process, image scanning; no keys in the pod, no route out but the allowlist

CONCLUSION

Not a replacement – a new class of work

A new class of work

Persistent, event-driven, tool-connected workflows. Durable responsibilities, delegated to agents.

Governed by OpenShift

NetworkPolicy, SCC, RBAC, Secrets, namespaces. The same primitives that run regulated workloads confine the agent.

Trust earned in stages

From read-only observation to privileged action. Capability grows only as evidence accumulates.

Pick one responsibility. Start it read-only. **Let the evidence move it up the ladder.**

